A Statistical Investigation of Kyiv Public Transport Distribution

KSE Probability & Statistics | Summer 2026

Nikita Bezverkhyi   Arsen Hanhalo   Sofia Tiutiushkina
Sofia Danilova

2026-07-13

This project focuses on researching the Kyiv public transport characteristics and planning. The main focus of it is to identify weak spots in the existing planning to shed light on the current situation and hopefully stimulate improvement. We research the planned intervals (headway), the transport capacity and differences between forms of public transportation.

Table of contents

Dataset: , « ». 1230 route × direction × time-slot rows constructed; **1036 analyzed** after excluding slots with fewer than 5 departures. 9 variables.

1 Introduction

1.1 Motivation and Context

As in any modern city, in Kyiv public transport is a vital part of a life of any citizen. And the goal of any city should be to make it as efficient as possible, because public transit plays a key part of the economic growth of a city. To put it simply the quicker and more efficient you can get a worker from point A to point B the more time they will spend working or the more free time they will have, in turn improving their productivity and mental state.

Because the practice of publicly available public transport data is quite rare for Ukrainian cities there are very few researches on this topic. And those ones which are conducted by city planners are often publicly unavailable. But we think that the public transportation is a very important topic for general public and there should be as much available information on it as possible, and this research is our contribution to that.

1.2 Dataset Description

The data used in this analysis were obtained from the Kyiv Open Data Portal. The dataset contains plan schedules collected directly from GTFS Static tables, containing routes, trips, and stop times, spanning a typical active weekday schedule. Each observation represents a unique route and direction combined with a specific operational time slot. Route–slot combinations with fewer than 5 scheduled departures were excluded from the analysis to reduce noise in interval estimates (1230 → 1036 observations).

1.3 Codebook

Variable name	Type	Unit / levels	Description
`route_id`	ID	unique character	Unique identifier for each transport route.
`transport_type`	Categorical	Tram / Bus / Trolleybus	Mode of public transport.
`time_slot`	Categorical	Peak Morning / Midday / Peak Evening / Night	Categorized operational hours of day.
`peak`	Categorical (logical)	TRUE / FALSE	Whether the slot is a rush-hour slot.
`n_vehicles`	Quantitative	Count	Number of distinct active vehicles assigned to the route.
`total_capacity`	Quantitative	Total passenger count	Combined vehicle passenger capacity.
`n_ctrl_points`	Quantitative	Count	Number of schedule control points on a route.
`sched_headway_min`	Quantitative	Minutes	Average scheduled interval between runs.
`sd_headway_min`	Quantitative	Minutes	Within-slot variability of the interval.
`n_departures`	Quantitative	Count	Number of departures in the slot.

1.4 Research Question and Hypotheses

> **!** Important
>
> **Research question:** Does the city evenly and rationally plan the allocation of the public transport — by time (day hours), transportation type and city districts?

The hypotheses we tested:

Note

H_1: Planned intervals during rush hours (7–10, 17–20) are smaller than non-rush hour intervals.

Note

H_2: Service intervals differ between 4 time slots (evening/night, midday, morning rush, evening rush) and three transportation types (tram, bus, trolleybus).

Note

H_3: The transport frequency category (frequent: 10 min, average: 10–20 min, rare: >20 min, during rush) does not depend on the transport type (Tram/Bus/Trolleybus).

Note

H_4: The transport capacity distribution is not balanced around the district population distribution.

Note

H_5: The service intervals are distributed exponentially (i.e., arrivals constitute a Poisson process).

2 Methodology

2.1 Data Preparation

Important

1. **Rows dropped:** we dropped all the weekend operations, because they would skew our data. Filtered only for the transport that is in use to not overestimate the vehicle number. Kept only the stops located in Kyiv, because this is the main area of our research. Excluded route–slot combinations with fewer than 5 departures (1230 → 1036 rows).
2. **Missing values:** we handled the missing values by omission to avoid the imputation bias. Used `na.rm = TRUE` to skip the missing values during aggregation. Because we used `lag()` to calculate the headways, the first departures created NA values

which were removed.
3. **Variable transformation:** normalized the time, created time intervals, standardised the `route_type` codes and `bodyType` strings into readable categories.
4. **Outlier handling:** set the timetable limits for values 0 and 120 minutes.

The full data-preparation pipeline is executed in `01_data_preparation.qmd`; the key steps are shown below for transparency (not re-evaluated here).

2.1.1 Load the data

```
library(curl)
urls <- c(
  gtfs_kyiv.zip =
    "https://data.kyivcity.gov.ua/dataset/rozklad-rukhu-miskoho-elektrychnoho-ta-avtomobilnol
  vehicles_kpt.json =
    "https://data.kyivcity.gov.ua/dataset/vidomosti-pro-transportni-zasoby-iaki-obsluhovuiut
  vehicles_marshrutky.csv =
    "https://data.kyivcity.gov.ua/dataset/vidomosti-pro-transportni-zasoby-iaki-obsluhovuiut
  stops_kpt.json =
    "https://data.kyivcity.gov.ua/dataset/dani-pro-mistse-rozmishchennia-zupynok-miskoho-elel
)
dir.create(here("data", "raw"), recursive = TRUE, showWarnings = FALSE)
for (nm in names(urls))
  curl::curl_download(urls[[nm]], here("data", "raw", nm))
```

2.1.2 Read the GTFS and weekday timetable

```
gtfs <- read_gtfs(here("data", "raw", "gtfs_kyiv.zip"))
weekday_services <- gtfs$calendar |> filter(monday == 1) |> pull(service_id)
```

2.1.3 Headways (planned intervals)

Headways are calculated at the first control point (`stop_sequence == 1` — verified in file; portal description mistakenly says “from 10”). These are terminal departure times; differences between consecutive departures = headways.

> ⚠ Warning
>
> The time in GTFS can be greater than 24:00 (departures after midnight) — so we check the NA values after conversion.

```
dep <- gtfs$stop_times |>
  filter(stop_sequence == 1) |>
  inner_join(gtfs$trips, by = "trip_id") |>
  filter(service_id %in% weekday_services) |>
  mutate(dep_sec = as.numeric(hms(departure_time)))

headways <- dep |>
  arrange(route_id, direction_id, dep_sec) |>
  group_by(route_id, direction_id) |>
  mutate(headway_min = (dep_sec - lag(dep_sec)) / 60) |>
  ungroup() |>
  filter(!is.na(headway_min), headway_min > 0, headway_min < 120) |>
  mutate(
    hour = dep_sec %/% 3600,
    time_slot = case_when(
      hour %in% 7:9   ~ "       ",
      hour %in% 10:16 ~ "  ",
      hour %in% 17:19 ~ "       ",
      TRUE            ~ "   /  ")) |>
  group_by(route_id, direction_id, time_slot) |>
  summarise(sched_headway_min = mean(headway_min),
            sd_headway_min = sd(headway_min),
            n_departures = n(), .groups = "drop")
```

Sanity check: verified 2026-07-12 on 5 routes (all 3 modes): tram 1 (4–9m), tram 11 (16–27m), tram 12 (11–17m — partial: direction 1 gives ~22–24m, likely incomplete feed), trolleybus 16 (16–20m), bus 49 (30m).

2.1.4 The route attributes

```
route_types <- gtfs$routes |>
  mutate(transport_type = case_when(
    route_type == 0  ~ "    ",
    route_type == 3  ~ "    ",
    route_type == 11 ~ "     ")) |>
```

```
  select(route_id, route_short_name, transport_type)

n_stops <- gtfs$stop_times |>
  group_by(trip_id) |> summarise(n = max(stop_sequence)) |>
  inner_join(gtfs$trips, by = "trip_id") |>
  group_by(route_id) |> summarise(n_ctrl_points = max(n))

veh_by_route <- fromJSON(here("data", "raw", "vehicles_kpt.json")) |>
  as_tibble() |>
  filter(status == "         ") |>
  separate_rows(routeId, sep = ",") |>
  mutate(routeId = str_trim(routeId),
         transport_type = case_when(
           str_detect(bodyType, "^    ")   ~ "    ",
           str_detect(bodyType, "^     ") ~ "     ",
           str_detect(bodyType, "^    ")   ~ "    ")) |>
  group_by(routeId, transport_type) |>
  summarise(n_vehicles = n_distinct(uid),
            total_capacity = sum(capacity, na.rm = TRUE), .groups = "drop")

route_types_veh <- route_types |>
  left_join(veh_by_route,
            by = c("route_short_name" = "routeId", "transport_type"))
```

2.1.5 Final analytical table

```
final <- headways |>
  left_join(route_types_veh, by = "route_id") |>
  left_join(n_stops, by = "route_id") |>
  mutate(peak = time_slot %in% c("       ", "       "))
write_csv(final, here("data", "processed", "analysis_table.csv"))
```

2.1.6 Data for H4: transportation capacity by district

Each stop in the city stop registry carries a KATOTTG district code, so no district polygons are needed; the 10 codes were decoded to district names (`data/raw/katottg_districts.csv`). GTFS control points are matched to districts via the nearest registry stop. A route's total capacity is split between districts proportionally to the share of its control points located in each district.

```
stops_kpt <- fromJSON(here("data", "raw", "stops_kpt.json")) |> as_tibble() |>
  filter(str_starts(CATUTTC, "UA80")) |>
  left_join(read_csv(here("data", "raw", "katottg_districts.csv")),
            by = c("CATUTTC" = "katottg"))

g_sf <- gtfs$stops |>
  mutate(across(c(stop_lat, stop_lon), as.numeric)) |>
  st_as_sf(coords = c("stop_lon", "stop_lat"), crs = 4326)
k_sf <- st_as_sf(stops_kpt, coords = c("lon", "lat"), crs = 4326)
gtfs_stops_d <- gtfs$stops |>
  mutate(district = stops_kpt$district[st_nearest_feature(g_sf, k_sf)])

capacity_by_district <- gtfs$stop_times |>
  inner_join(gtfs$trips, by = "trip_id") |>
  distinct(route_id, stop_id) |>
  left_join(gtfs_stops_d |> select(stop_id, district), by = "stop_id") |>
  count(route_id, district) |>
  group_by(route_id) |> mutate(share = n / sum(n)) |> ungroup() |>
  left_join(route_types_veh |> select(route_id, total_capacity),
            by = "route_id") |>
  group_by(district) |>
  summarise(capacity = sum(total_capacity * share, na.rm = TRUE))
write_csv(capacity_by_district,
          here("data", "processed", "capacity_by_district.csv"))
```

2.2 Exploratory Data Analysis

```
d |> group_by(peak) |>
  summarise(n = n(), median = median(sched_headway_min),
            mean = mean(sched_headway_min), sd = sd(sched_headway_min)) |>
  knitr::kable(digits = 2,
               caption = "Summary statistics for the analytical sample.")
```

Table 1: Summary statistics for the analytical sample.

peak	n	median	mean	sd
FALSE	526	26.56	28.82	12.98
TRUE	510	22.80	22.56	8.37

Well both median and mean waiting time periods are quite big. Also we see that standard deviation for non-peak time is high. Personally I would not trust the transport if the median waiting time is 23 minutes at the rush hour.

```
ggplot(d, aes(sched_headway_min)) +
  geom_histogram(bins = 40) +
  labs(title = "General distribution of the intervals",
       x = "Interval, min", y = "Num") +
  theme_minimal()
```

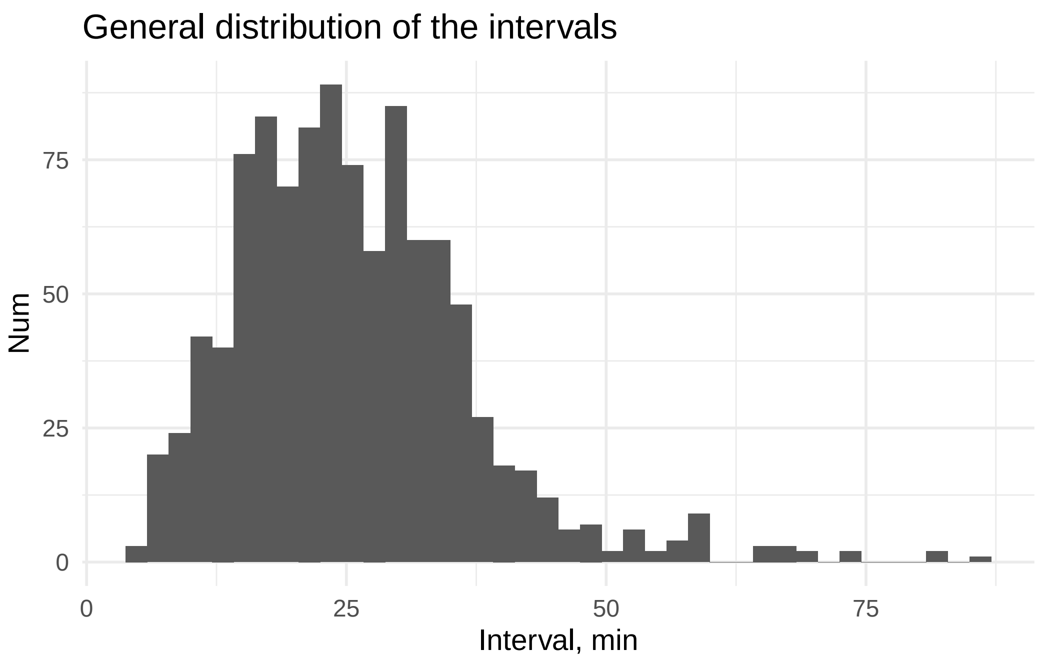


Figure 1: The interval distribution is right-skewed with a heavy tail.

```
ggplot(d, aes(time_slot, sched_headway_min)) +
  geom_boxplot() +
  labs(x = "Time slot", y = "Interval, min") +
  theme_minimal()
```

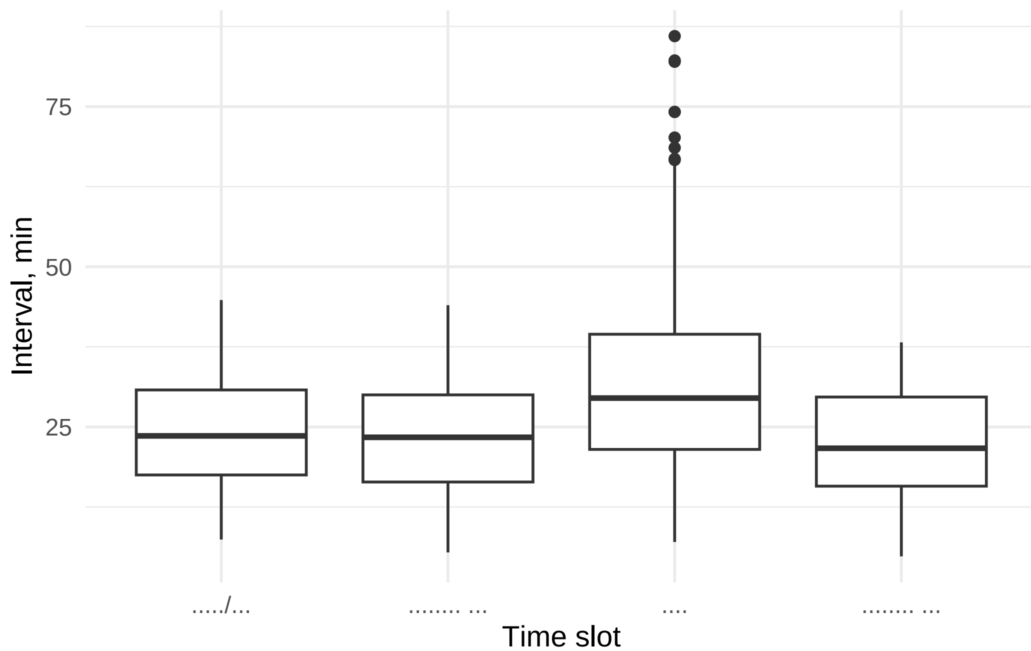


Figure 2: Intervals by time slot: midday stands out with the largest and most variable intervals.

```
ggplot(d, aes(transport_type, sched_headway_min, color = transport_type)) +
  geom_boxplot() +
  labs(x = "Transport type", y = "Interval, min") +
  theme_minimal() +
  theme(legend.position = "none")
```

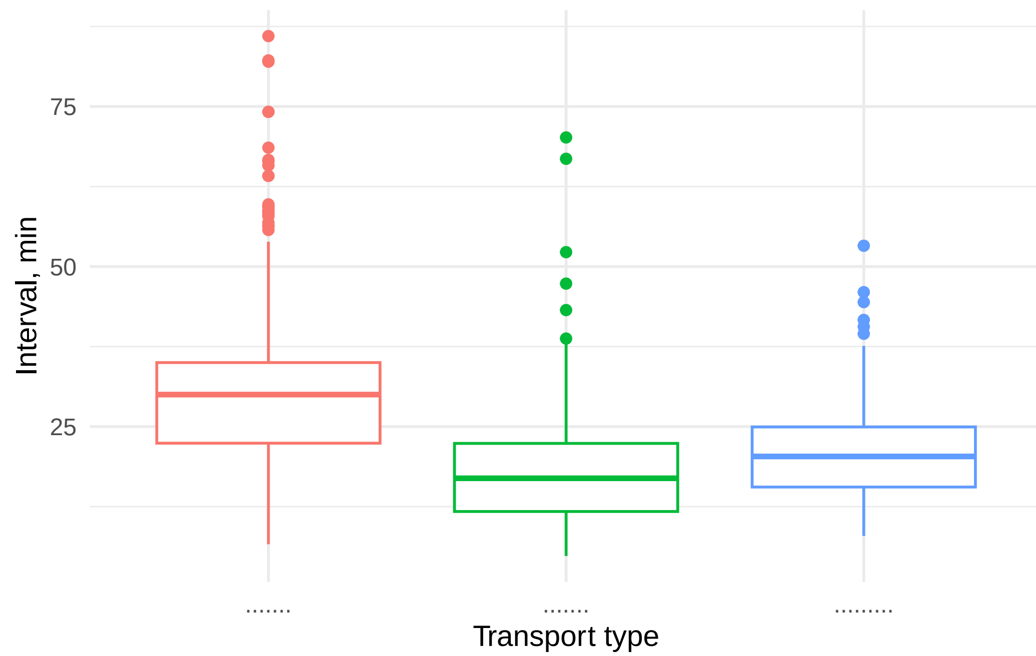


Figure 3: Intervals by transport type: trams hold the tightest schedules.

2.3 Statistical Methods

For this research we used the following statistical methods:

1. **Mann–Whitney U Test, one-sided (H1: Peak vs. Off-Peak).** Compares the distribution of a continuous variable (headway) between two independent groups (peak/off-peak). The one-sided alternative is used because H_1 is directional (intervals are *smaller* during rush hours). *Justification:* Shapiro–Wilk and QQ-plots rejected normality in both groups, so the rank-based alternative to the parametric t-test is required.
2. **Kruskal–Wallis Test (H2: Temporal Slots & Transport Modes).** Compares three or more groups within the Mann–Whitney framework. *Justification:* normality is rejected and group variances differ, so a rank-based method is used; Holm adjustment was applied in post-hoc pairwise comparisons to prevent false positives.
3. **Chi-Square (χ^2) Test of Independence (H3: Mode vs. Frequency).** Tests the statistical association between two categorical variables. *Justification:* one expected cell frequency was below 5, so we used a Monte-Carlo simulated p-value instead of the asymptotic approximation.
4. **Chi-Square (χ^2) Goodness-of-Fit Test (H4: Spatial Equity).** Evaluates if a sample distribution matches a theoretical multinomial baseline (district population shares). Using it we can identify which districts are over-served or under-served by transit.

5. **Kolmogorov–Smirnov Test (H5: Stochastic Modeling).** Compares the sample empirical distribution (ECDF) against a theoretical continuous CDF. *Justification:* the data is continuous, so no artificial binning is needed. Because the rate parameter is estimated directly from the data, the p-value acts as an approximation.

3 Results

3.1 Main Findings

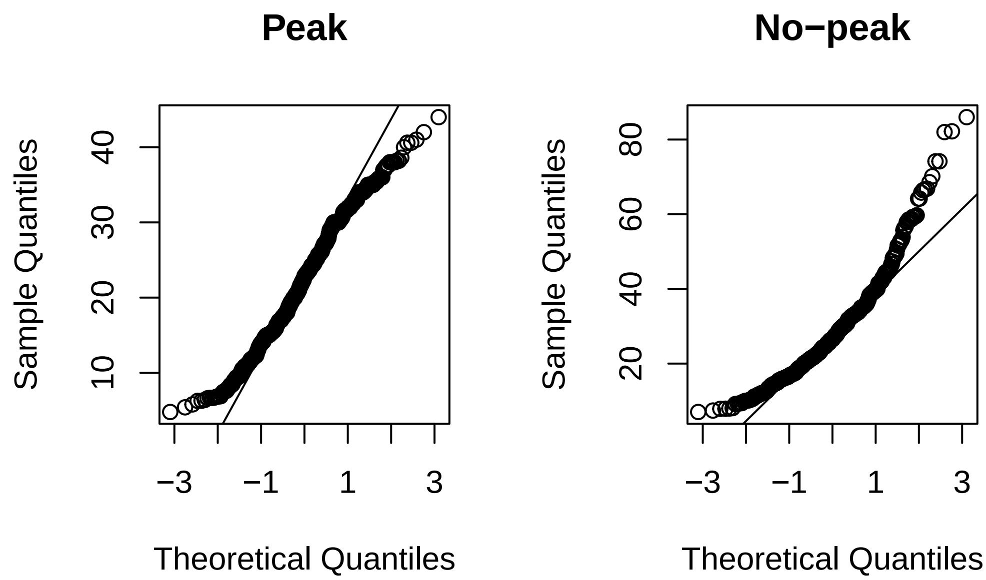
> **Note**
>
> H_1: **Planned intervals during rush hours (7–10, 17–20) are smaller than non-rush hour intervals.**

```r
d |> group_by(peak) |>
  summarise(shapiro_p = shapiro.test(sched_headway_min)$p.value) |>
  knitr::kable(caption = "Shapiro-Wilk normality test p-values by peak status.")
```

Table 2: Shapiro-Wilk normality test p-values by peak status.

peak	shapiro_p
FALSE	0.00e+00
TRUE	2.36e-05

```r
par(mfrow = c(1, 2))
qqnorm(d$sched_headway_min[d$peak == TRUE],  main = "Peak")
qqline(d$sched_headway_min[d$peak == TRUE])
qqnorm(d$sched_headway_min[d$peak == FALSE], main = "No-peak")
qqline(d$sched_headway_min[d$peak == FALSE])
```




```
par(mfrow = c(1, 1))
```

Both groups deviate clearly from normality (heavy right tails on the QQ-plots; Shapiro–Wilk p-values far below 0.05), which justifies the rank-based test instead of the t-test.

```
# One-sided test, because H1 is directional ("smaller during rush").
# Group order in the formula is FALSE (off-peak) then TRUE (peak),
# so alternative = "greater" states: off-peak intervals are stochastically
# GREATER than peak intervals, i.e. peak intervals are smaller.
h1_test <- wilcox.test(sched_headway_min ~ peak, data = d,
                       alternative = "greater")
h1_test
```

```
    Wilcoxon rank sum test with continuity correction

data:  sched_headway_min by peak
W = 170503, p-value = 2.103e-14
alternative hypothesis: true location shift is greater than 0
```

The H_0 was rejected in favour of the one-sided alternative (p = 2.1e-14): rush-hour intervals are systematically smaller. The effect is also visible descriptively — the median scheduled interval is 22.8 min during rush hours versus 26.6 min off-peak.

> **Note**
>
> H_2: **Service intervals differ between 4 time slots (evening/night, midday, morning rush, evening rush) and three transportation types (tram, bus, trolleybus).**

```r
kruskal.test(sched_headway_min ~ factor(time_slot), data = d)
```

```

	Kruskal-Wallis rank sum test

data:  sched_headway_min by factor(time_slot)
Kruskal-Wallis chi-squared = 92.385, df = 3, p-value < 2.2e-16
```

```r
kruskal.test(sched_headway_min ~ factor(transport_type), data = d)
```

```

	Kruskal-Wallis rank sum test

data:  sched_headway_min by factor(transport_type)
Kruskal-Wallis chi-squared = 249.25, df = 2, p-value < 2.2e-16
```

```r
pairwise.wilcox.test(d$sched_headway_min, d$time_slot,
                     p.adjust.method = "holm")
```

```

	Pairwise comparisons using Wilcoxon rank sum test with continuity correction

data:  d$sched_headway_min and d$time_slot

               /
       0.34      -             -
          5.4e-09   6.3e-13       -
       0.03      0.34          < 2e-16

P value adjustment method: holm
```

```r
pairwise.wilcox.test(d$sched_headway_min, d$transport_type,
                     p.adjust.method = "holm")
```

```

	Pairwise comparisons using Wilcoxon rank sum test with continuity correction
```

```
data:  d$sched_headway_min and d$transport_type


       < 2e-16 -
      < 2e-16 1.4e-05

P value adjustment method: holm
```

The H_0 was rejected for both factors — there is a difference in both time slots and transport types. Here we can see that there are no difference between evening and evening peak and morning and evening peak. But we have high difference between day and late evening, day and evening peak and between day and morning peak. We also have very small difference between late evening and morning peak, which is the most confusing result we obtained.

Moreover there is more difference between tram and trolleybus than between bus and either of other transport types.

```r
d |> group_by(time_slot) |>
  summarise(median = median(sched_headway_min),
            mean = mean(sched_headway_min),
            sd = sd(sched_headway_min)) |>
  knitr::kable(digits = 2, caption = "Interval summary by time slot.")
```

Table 3: Interval summary by time slot.

time_slot	median	mean	sd
/	23.60	24.35	8.49
	23.38	23.01	8.54
	29.50	32.09	14.63
	21.67	22.14	8.21

With this summary table we can see the difference between the day and every other time-slot. Midday has the highest standard deviation — 14.6, median — 29.5 and mean — 32.09 values, while other time-slots have more or less the same values. So the real gap is not between peak and non-peak but between the day and everything else. Moreover, higher standard deviation at the daytime tells us about an unstable schedule at this period of time compared to the rest of the day.

```r
d |> group_by(transport_type) |>
  summarise(median = median(sched_headway_min)) |>
  arrange(median) |>
  knitr::kable(digits = 2, caption = "Median interval by transport type.")
```

Table 4: Median interval by transport type.

transport_type	median
	16.92
	20.33
	30.00

Here is a just-to-compare table to see that the tram is the best-planned transport per our investigation. Nevertheless we should mention there are way more bus observations than other types of transport (590 bus rows vs. 125 tram and 321 trolleybus rows), and that particular type of transport needs more attention from the city transport administration.

> **Note**
>
> H_3: **The transport frequency category (frequent: 10 min, average: 10–20 min, rare: >20 min, during rush) does not depend on the transport type (Tram/Bus/Trolleybus).**

```
peak_data <- d |> filter(peak) |>
  mutate(freq_cat = cut(sched_headway_min, breaks = c(0, 10, 20, Inf),
                        labels = c("    ", "     ", "      ")))
tab3 <- table(peak_data$transport_type, peak_data$freq_cat)
tab3
```

```
          10        75        204
          16        27         18
         10        73         77
```

```
chi3 <- chisq.test(tab3, simulate.p.value = TRUE, B = 10000)
round(chi3$expected, 2)
```

```
       20.40     99.17    169.43
        4.31     20.93     35.76
      11.29     54.90     93.80
```

```
chi3
```

```
	Pearson's Chi-squared test with simulated p-value (based on 10000
	replicates)

data:  tab3
X-squared = 69.709, df = NA, p-value = 9.999e-05
```

The H_0 was rejected: trams are used more frequently while buses are used less frequently, but there are a bigger amount of them.

The high chi-square value of 69.7 shows us that the difference in the intervals for the three modes of transportation across categories is very significant and differs greatly from proportionally equal values; for if Kyiv had planned the intervals for buses, trams, and trolleybuses to be strictly the same, the value would have been close to 0. The expected frequency for trams in the "frequent" category turned out to be less than the threshold ($4.31 < 5$). Therefore, the standard approximation to the chi-square distribution could give us a misleading value. For this reason, we used a p-value simulation: the computer independently generates a distribution from 10000 random combinations of the available data, and the degrees of freedom in this case are meaningless — therefore, the results show NA. The simulated p-value (~1e-04) is very small and shows us that the chance of such a transportation distribution occurring by chance is very low.

An analysis of this data confirmed that trams have the highest scheduled arrival frequency during peak hours; however, since Kyiv's tram network is smaller than its bus and trolleybus networks, this result does not allow us to draw broad conclusions about the superiority of one mode of transportation over another. However, we can draw a specific conclusion that provides a clear practical recommendation for passengers: if there is a choice between a bus stop and a tram stop in the same direction, it is always better to choose the tram.

> ℹ Note
>
> H_4: **The transport capacity distribution is not balanced around the district population distribution.**

```r
cap_d <- read_csv(here("data", "processed", "capacity_by_district.csv"))
pop   <- read_csv(here("data", "raw", "population_districts.csv"))

d4 <- inner_join(cap_d, pop, by = "district")
test4 <- chisq.test(x = d4$capacity, p = d4$population / sum(d4$population))
test4
```

```
	Chi-squared test for given probabilities

data:  d4$capacity
```

```
X-squared = 53820, df = 9, p-value < 2.2e-16
```

```
d4 <- d4 |> mutate(stdres = round(test4$stdres, 1)) |> arrange(stdres)
d4 |> knitr::kable(
  caption = "Observed transport capacity, population, and standardized residuals by district
```

Table 5: Observed transport capacity, population, and standardized residuals by district.

district	capacity	population	stdres
	34943.93	347512	-102.4
	42349.77	340399	-65.0
’	51383.26	386496	-56.0
	19607.48	165360	-49.9
	49561.31	316104	-15.3
	44838.42	253124	12.7
	66179.41	364887	22.2
	65852.44	356190	27.4
	55718.06	210425	113.5
	63202.91	210205	155.1

The H_0 (capacity proportional to population) was rejected: there is a two-way disbalance — huge residential districts are lacking public transport while smaller central districts have an excess of transport volume.

```
metro <- read_csv(here("data", "processed", "metro_stations_by_district.csv"))
d4m <- d4 |> left_join(metro, by = "district") |>
  mutate(metro_per_100k = metro_stations / population * 1e5)
d4m |> select(district, stdres, metro_stations, metro_per_100k) |>
  knitr::kable(digits = 2,
    caption = "Standardized residuals and metro accessibility by district.")
```

Table 6: Standardized residuals and metro accessibility by district.

district	stdres	metro_stations	metro_per_100k
	-102.4	7	2.01
	-65.0	4	1.18
’	-56.0	3	0.78
	-49.9	10	6.05
	-15.3	4	1.27

district	stdres	metro_stations	metro_per_100k
	12.7	8	3.16
	22.2	2	0.55
	27.4	3	0.84
	113.5	3	1.43
	155.1	8	3.81

```
cor(d4m$stdres, d4m$metro_per_100k, method = "spearman")
```

```
[1] 0.07878788
```

```
d4m |>
  ggplot(aes(x = reorder(district, stdres), y = stdres, fill = stdres > 0)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = paste0(" : ", metro_stations),
                y = ifelse(stdres > 0, stdres + 14, stdres - 14)), size = 3.4) +
  scale_fill_manual(values = c("firebrick", "steelblue")) +
  coord_flip() +
  labs(title = "                                 ",
       subtitle = "              chi-square    : «-»         , «+»        .\n : n -
       x = NULL, y = "             ",
       caption = "  :             (GTFS,      ,       );       -            .   , 2022") +
  theme_minimal(base_size = 13)
```

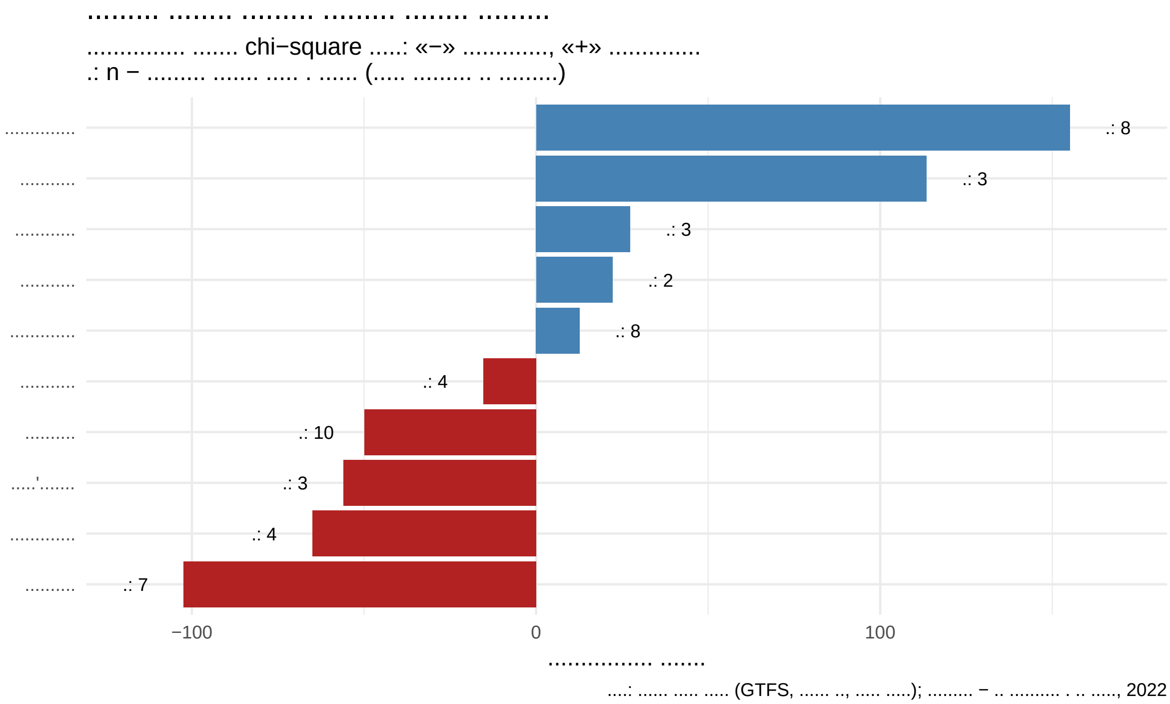


Figure 4: Two-way disbalance of planned surface capacity relative to population; metro does not compensate.

We evaluated whether the spatial distribution of planned surface transport passenger capacity across Kyiv's districts matches their respective population distributions (chi-square goodness-of-fit; population serves as a demand proxy since ticket validation data is not publicly available). The hypothesis of proportionality was soundly rejected ($\chi^2 = 53{,}820$, $df = 9$, $p < 2.2 \times 10^{-16}$), revealing a bidirectional imbalance. Large residential districts are severely underserved — Darnytskyi accounts for 11.8% of the city's population but receives only 7.1% of surface capacity ($stdres = -102.4$), followed closely by Sviatoshynskyi (-65.0) and Solomianskyi (-56.0). Conversely, the central Shevchenkivskyi ($+155.1$) and Podilskyi ($+113.5$) districts, each housing ~7% of the population, receive 11–13% of the total capacity.

We tested whether this disparity is offset by the underground rapid transit network: Spearman's rank correlation between the test residuals and the number of metro stations per 100,000 residents is a mere 0.08, indicating no systematic compensation. Sviatoshynskyi and Solomianskyi operate at a structural deficit across both transit systems; the sole exception is Pecherskyi, whose surface transit deficit is offset by the highest metro density in the city. Notably, given a sample size of this magnitude (hundreds of thousands of passenger slots), even a minor deviation from proportionality would yield statistical significance; thus, the true analytical insight lies not in the p-value itself, but in the sign and magnitude of the standardized residuals.

> ℹ Note
>
> H_5: **The service intervals are distributed exponentially (i.e., arrivals constitute a Poisson process).**

```
for (type in c("    ", "    ", "     ")) {
  intervals <- d |>
    filter(transport_type == type, peak == TRUE) |>
    pull(sched_headway_min) |> na.omit()
  lambda_est <- 1 / mean(intervals)
  ks_res <- ks.test(intervals, "pexp", rate = lambda_est)
  cat(sprintf("%-10s n = %3d  lambda = %.4f  D = %.4f  p = %.3g\n",
              type, length(intervals), lambda_est,
              ks_res$statistic, ks_res$p.value))
}
```

```
     n = 289  lambda = 0.0394  D = 0.3565  p = 2.53e-32

     n =  61  lambda = 0.0634  D = 0.2785  p = 0.000155

      n = 160  lambda = 0.0499  D = 0.3394  p = 1.97e-16
```

The H_0 (exponentially distributed intervals, i.e. fully random arrivals) was rejected for all three modes. The D-statistic ranges from 0.279 for trams to 0.357 for buses (trolleybuses: 0.339). This value indicates the discrepancy between the scheduled intervals and the ideal random curve: if the vehicles arrived at stops in a completely random and independent manner, the values would be close to 0. The p-value for all modes is far below 0.05 (the largest is 1.55e-04 for trams). This shows that the probability that such a distribution of intervals occurred by chance is practically zero: the schedules are genuinely planned rather than random. Buses and trolleybuses show the greatest deviation from randomness, indicating that fixed intervals are maintained more strictly there. Since we used data from the transit plan rather than actual arrival times, we cannot definitively state whether public transit schedules can be trusted in reality, but we can offer a practical recommendation: planned intervals are structured and predictable, so checking the schedule is worthwhile.

3.2 Main figure

```r
ggplot(d, aes(x = time_slot, y = sched_headway_min, fill = transport_type)) +
  geom_violin(alpha = 0.4, color = NA) +
  geom_boxplot(width = 0.18, outlier.alpha = 0.35, fill = "white") +
  facet_wrap(~transport_type) +
  scale_fill_manual(values = c("    " = "#e07a7a",
                               "    " = "#63b56f",
                               "     " = "#7aa6e0")) +
  guides(fill = "none") +
  labs(title = "                        ,                          ",
       subtitle = "                                          ",
       x = "      ", y = "             (   )",
       caption = "    :             GTFS-     «      » (2026)") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

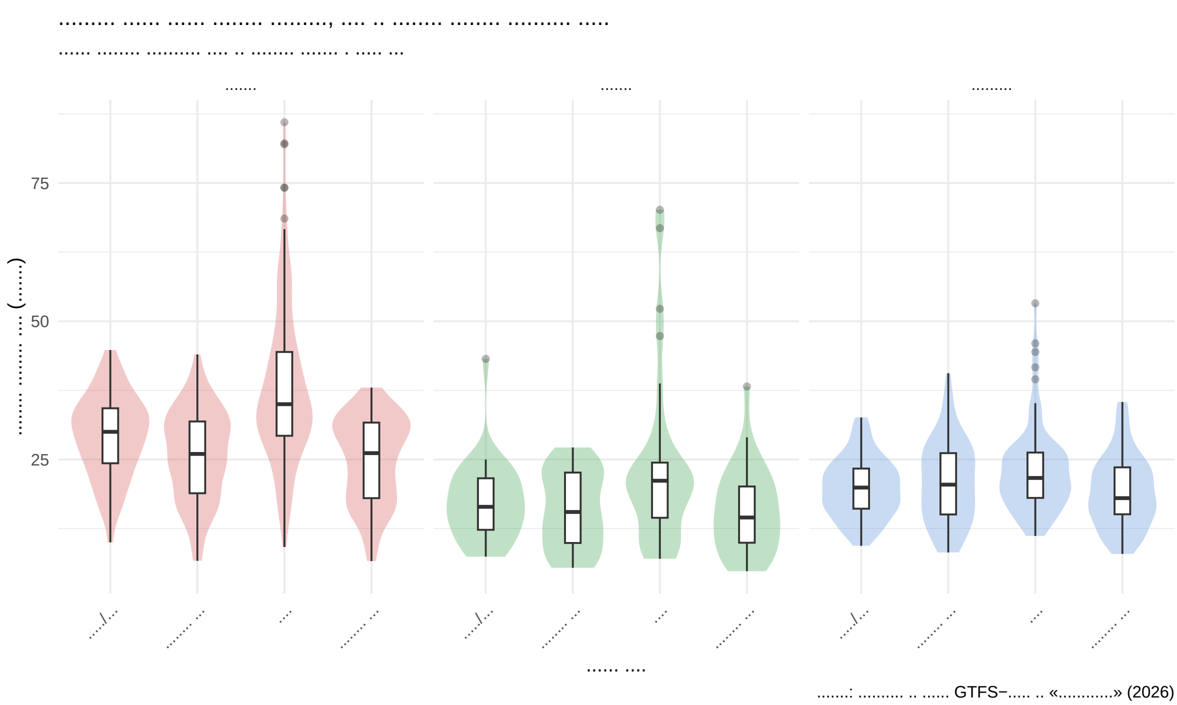


Figure 5: Planned headways by time slot and transport mode (weekdays).

3.3 Bonus: Plan vs. Reality Pilot

The core of this project analyses the *planned* schedule. As a bonus pilot we collected 362 real-time GPS snapshots of the whole fleet (Sunday 09:58 — Monday 09:17, one snapshot every 2 minutes; 164,451 vehicle positions) and checked, on the same three routes used for methodology validation, how the actual Monday morning rush compares to the plan. An "arrival" is a vehicle crossing into a 200 m radius around the route terminal; the inside/outside state resets only after the vehicle moves 450 m away (hysteresis), so a vehicle parked at the terminal is not double-counted. The 2-minute polling grid adds ±2 min of noise per event.

```
library(geosphere)   # install.packages("geosphere") if missing
rt <- read_csv(here("data", "processed", "realtime_positions.csv")) |>
  mutate(dt = as_datetime(timestamp, tz = "Europe/Kyiv"))

pilots <- c("1_1", "2_12", "3_100")
trips_g      <- read_csv(here("data", "raw", "gtfs", "trips.txt"))
stop_times_g <- read_csv(here("data", "raw", "gtfs", "stop_times.txt"))
stops_g      <- read_csv(here("data", "raw", "gtfs", "stops.txt"))

# main terminal of each direction = most frequent first control point
terminals <- stop_times_g |>
  filter(stop_sequence == 1) |>
  inner_join(trips_g, by = "trip_id") |>
  filter(route_id %in% pilots) |>
  count(route_id, direction_id, stop_id, sort = TRUE) |>
  group_by(route_id, direction_id) |> slice(1) |> ungroup() |>
  left_join(stops_g, by = "stop_id")

arrivals_at <- function(rt_route, term_lat, term_lon) {
  rt_route |>
    arrange(vehicle_id, timestamp) |>
    mutate(dist = distHaversine(cbind(lon, lat), c(term_lon, term_lat))) |>
    group_by(vehicle_id) |>
    mutate(inside = purrr::accumulate(
             dist, .init = FALSE,
             \(state, dd) if (dd < 200) TRUE else if (dd > 450) FALSE else state)[-1],
           arrival = inside & !lag(inside, default = FALSE)) |>
    ungroup() |> filter(arrival) |> arrange(dt) |> pull(dt)
}

win <- rt |> filter(dt >= ymd_hm("2026-07-13 06:45", tz = "Europe/Kyiv"))

results <- list()
for (i in seq_len(nrow(terminals))) {
  t <- terminals[i, ]
  arr <- arrivals_at(win |> filter(route_id == t$route_id),
                     t$stop_lat, t$stop_lon)
```

```
  hw <- diff(as.numeric(arr) / 60)
  hw <- hw[hw > 2 & hw < 90]
  results[[i]] <- tibble(route_id = t$route_id,
                         direction_id = t$direction_id,
                         n_arrivals = length(arr),
                         fact_mean = mean(hw), fact_median = median(hw),
                         headways = list(hw))
}
actual <- bind_rows(results)

plan_pilot <- d |>
  filter(route_id %in% pilots, time_slot == "       ") |>
  group_by(route_id, direction_id) |>
  summarise(plan_headway = mean(sched_headway_min), .groups = "drop")

actual |>
  left_join(plan_pilot, by = c("route_id", "direction_id")) |>
  select(-headways) |>
  knitr::kable(digits = 1,
    caption = "Actual vs. planned morning-rush headways, Monday 2026-07-13.")
```

Table 7: Actual vs. planned morning-rush headways, Monday 2026-07-13.

route_id	direction_id	n_arrivals	fact_mean	fact_median	plan_headway
1_1	0	25	7.6	7.6	4.8
1_1	1	28	7.3	6.3	5.8
2_12	0	4	30.2	12.9	13.6
2_12	1	9	16.9	15.8	16.0
3_100	0	2	37.7	37.7	30.0
3_100	1	2	31.4	31.4	30.0

```
hw_tram1 <- actual |> filter(route_id == "1_1") |> pull(headways) |> unlist()
plan_tram1 <- plan_pilot |> filter(route_id == "1_1") |>
  summarise(m = mean(plan_headway)) |> pull(m)
wilcox.test(hw_tram1, mu = plan_tram1, alternative = "greater")
```

```

	Wilcoxon signed rank test with continuity correction

data:  hw_tram1
V = 531, p-value = 0.003462
alternative hypothesis: true location is greater than 5.294737
```

Trolleybus 16 matched its planned headway almost exactly (median 15.8 min vs. planned 16.0 at the “vul. Mrii” terminal), while tram 1 ran markedly less frequently than planned (7.3–7.6 min vs. planned 4.8–5.8; one-sided Wilcoxon signed-rank against the planned value: $p = 0.0035$). Bus 49 produced too few events (2 per direction) for any conclusion. Ties in the Wilcoxon test stem from the 2-minute time quantization, so the normal approximation with continuity correction is used.

4 Discussion

4.1 Summary

We find a statistically significant, structurally critical difference in public transport intervals across times of day and modes of transport, and a pronounced two-way spatial imbalance: large residential districts (above all Darnytskyi) receive far less planned surface capacity than their population share, while central districts receive far more — and the metro network does not compensate for this gap. This directly answers our research question: Kyiv’s planned transport grid is unevenly distributed, showing exceptional vulnerabilities during off-peak midday operation blocks and in the largest residential districts.

4.2 Interpretation and Contextualisation

The practical impact of these variations is extensive. The massive shift in standard deviation up to 14.6 minutes during midday reveals an erratic supply model that directly harms mid-day commuters. Trams maintain the tightest, most consistent planning priority across the municipal system layout. On the spatial side, the residual structure of the H4 test provides a concrete, data-backed priority list for transit planners: Darnytskyi, Sviatoshynskyi and Solomianskyi are systematically under-provisioned even after accounting for metro access.

4.3 Limitations and Self-Critique

1. **Scheduled vs. actual reality:** this project investigates GTFS Static parameters, tracking planned intents rather than accounting for actual real-time congestion delays.
2. **Demand proxies:** we rely on population distributions as a demand baseline, which does not map variable daily commuting footprints; the 2022 population structure may also differ from the current wartime one.
3. **Coverage:** the GTFS feed covers only the municipal operator («   »); private minibuses are absent, and known feed gaps exist (e.g. missing runs on tram 12, confirmed by citizen requests on the data portal).
4. **Capacity accounting:** a vehicle assigned to several routes is counted in each of them, so `total_capacity` reflects capacity available to a route rather than the unique city fleet.

5. **Temporal scope:** the real-time pilot (see *Bonus: Plan vs. Reality Pilot*) suggests the plan is meaningful but not guaranteed: trolleybus 16 matched its planned morning-rush headway almost exactly, while tram 1 ran markedly less frequently than planned (7.3–7.6 min vs. 4.8–5.8 min, Wilcoxon $p = 0.0035$). Still, one morning of data limits generalizability, and the 2-minute polling grid adds ±2 min noise per event.

4.4 What We Would Do Differently

If we had more time, we would deploy active servers to parse automated daily scripts for two weeks, capturing true live metrics against static baseline assumptions.

References

1. — **GTFS Static.** , / « ». https://data.kyivcity.gov.ua/dataset/rozklad-rukhu-miskoho-elektrychnoho-ta-avtomobilnoho-transportu-dep-transport
2. (,). https://data.kyivcity.gov.ua/dataset/vidomosti-pro-transportni-zasoby-iaki-obsluhovuiut-pasazhyrski-avtobusni-troleibusni-t-dep-transport
3. (,). https://data.kyivcity.gov.ua/dataset/dani-pro-mistse-rozmishchennia-zupynok-miskoho-elektrychnoho-ta-avtomobilnoho-transpor-dep-transport
4. (`underground` « ») — 52 .
5. **. , 2022.** . . http://www.kyiv.ukrstat.gov.ua/p.php3?c=1123&lang=1

5 Appendix

5.1 Team Contributions

Team member	Contribution
Arsen Hanhalo	Data pipeline (GTFS, vehicles, stops), headway computation and validation, H4 (district capacity, metro robustness check).
Sofia Danilova	Exploratory data analysis, H1 and H2 (diagnostics, Mann–Whitney, Kruskal–Wallis with post-hoc).

Team member	Contribution
Sofia Tiutiushkina	H3 (chi-square independence with Monte-Carlo p-value) and H5 (Kolmogorov–Smirnov).
Nikita Bezverkhyi	Report assembly and editing, presentation slides, realtime GPS collection support.

All team members have reviewed the full code and analysis and take collective responsibility for the report and presentation.

5.2 Use of AI

Code skeletons, project structure and data-source discovery were assisted by Claude (Anthropic); all code was reviewed, adapted and executed by the team, and the statistical interpretations and narrative were written by the authors. AI-assisted code is disclosed here in accordance with the course policy on generative AI.